

Solar Elevation Estimation from Single Images Using Convolutional Neural Networks for Multi-Image Geolocation

[Alex Nowlan]

Department of Computer Science and Software Engineering
University of Canterbury, Christchurch, New Zealand
[ano58@uclive.ac.nz]

Abstract—[TODO: Write the abstract last. Cover: (1) problem statement – estimating geographic location from consumer photographs without GPS metadata; (2) your approach – fine-tuning an ImageNet-pretrained CNN (ResNet/VGG/etc.) to regress solar elevation angle from a single image, combined with an astronomical sun-position model to infer latitude/longitude from a set of temporally spaced, co-located images; (3) key quantitative result, e.g. “median location error of X km on our test set”; (4) significance – passive, metadata-free geolocation applicable to archival or forensic photography.]

Index Terms—solar elevation, geolocation, convolutional neural network, transfer learning, sun position model, ImageNet, image forensics

I. INTRODUCTION

[TODO: Write the introduction following the five-paragraph structure above. Be specific: name the CNN backbone you chose and why (e.g. ResNet-50 – good accuracy/speed trade-off, widely benchmarked). Mention that timestamps must be available in EXIF or supplied manually.]

The automatic determination of where a photograph was taken — without relying on embedded GPS metadata — is an open problem in computer vision with applications in digital forensics [7], investigative journalism, and autonomous navigation. [TODO: Expand this opening paragraph and add real citations.]

II. RELATED WORK

A. Image-Based Geolocation

[TODO: Discuss landmark / scene-matching approaches (e.g. IM2GPS [3], PlaNet [4]). Note they require large retrieval databases and fail on featureless outdoor scenes.]

B. Solar Cues in Computer Vision

[TODO: Cover work that uses shadows or sky appearance to estimate sun direction (e.g. hold-geoffroy2017 deep sky model, lalonde2012 outdoor illumination). Contrast with your regression-based, end-to-end CNN approach.]

C. Transfer Learning for Regression

[TODO: Briefly explain why fine-tuning an ImageNet-pretrained network for a regression target (elevation angle in degrees) is sound practice. Reference a general transfer learning paper and one domain-specific example.]

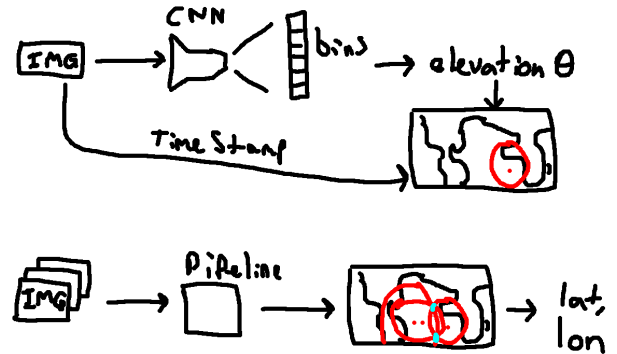


Fig. 1: System pipeline. A CNN estimates solar elevation $\hat{\theta}$ from each input image. Given timestamps, the sun-position model inverts the elevation to candidate (latitude, longitude) sets. Multiple images constrain the solution to a unique geographic location.

D. Astronomical Sun-Position Models

[TODO: Cite the algorithm you use (e.g. NREL Solar Position Algorithm, Grena 2012, or pysolar). Note accuracy (sub-0.01° for SPA).]

III. METHODOLOGY

A. System Overview

The timestamp of an input image is used to calculate the subsolar-point (SP), the location on the globe where the sun is directly overhead for that time. A Convolutional Neural Network (CNN) is used to estimate the solar elevation of an image. Using the solar elevation of the sun in the scene, and the SP for the given image, a circle of possible locations can be derived for where the image must have been taken. Multiple candidate circles can be derived when given multiple images from the same location at different times of year. The intersection of these candidate sets can be taken to arrive at 2 unique points on the globe where the image must be taken from, mirrored across the equator.

B. CNN Architecture and Transfer Learning

The image classifier model, EfficientNet [cite], was used as a backbone architecture for determining solar elevation from

images. A pre-trained version of this CNN for the purpose of image classification using the ImageNet-1K dataset was used, as important feature recognition methods such as edge detection and shadow detection are backed into the pre-trained weights. These fetures are important when determining solar elevation.

The original classifier head was replaced with a feature head of 32 bins.

The backbone network f_ϕ maps an input image $\mathbf{I} \in \mathbb{R}^{H \times W \times 3}$ to one of 32 bins, each representing $\frac{90}{32} = 2.81$ degrees for solar elevation:

$$\hat{\beta} = f_\phi(\mathbf{I}), \quad \hat{\beta} \in [0, 32]. \quad (1)$$

[TODO: Explain the training objective, e.g. $\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (\hat{\theta}_i - \theta_i)^2$.]

During training, Loss is calculated using KL Divergence, which measures how different two probability distributions are. The Loss function can be defined as:

$$Loss = \Sigma(P(target) * \log(\frac{P(target)}{Q(predicted)})) \quad (2)$$

During inference, predicted solar elevation can be extracted from the bins by taking the expected value of the distribution as follows:

$$\hat{\theta} = \Sigma(\text{softmax(logits)}[i] * \text{binCenter}[i]). \quad (3)$$

This enables the model to be able to predict a smooth output for elevation rather than 32 discreat bins.

A full training overview is shown in table 1.

C. Dataset and Ground-Truth Labels

[TODO: Describe:

- **Where your training images came from (Flickr, personal collection, public dataset). State the total number and train/val/test split.**
- **How ground-truth elevation angles were computed from EXIF timestamps and known GPS coordinates using your sun-position library.**
- **Any data augmentation applied (random crop, colour jitter, etc.).**
- **Class / angle distribution (histogram would be great as a figure).**

]

A dataset consisting of 6467 equirectangular panorama images taken from the Google street view API was manually labels for solar elevation. The labelling split the set into two categories:

- No Visible Sun
- Visible sun + manually labeled elevation

This resulted in 2404 clean images with ground truth labels.

Each panorama was used to create 12 training images with the following randomized parameters:

- Feild of View. $FOV \in [50, 90]$.
- Yaw $\in [0, 360]$ degrees.
- Pitch $\in [-30, 30]$ degrees from horizon line.

The final training set consisted of 28,848 images labeled with solar elevation.

1,000 of these images were kept out of traing and used for evaluation resulting in a total training set size of 27,848 images.

The ground truth for the training set is converted from degrees to a gaussian with sigma=2.5 centered at the closest bin. This results in a soft label for each image where the probability mass is spread across roughly ± 5 degrees of the true elevation.

D. Sun-Earth Geometric Model

[TODO: Explain the astronomical model you use to compute solar elevation θ given observer latitude ϕ , longitude λ , date, and UTC time t :

$$\theta = g(\phi, \lambda, t), \quad (4)$$

where $g(\cdot)$ is the Solar Position Algorithm (or equivalent). Cite the algorithm. Note the precision achievable (< 0.01 for SPA).]

The elevation of the sun, paired with an accurate timesamp of when such elevation is measured provides a narrow search range for the location of where such an elevation was measured.

The subsolar point can be easilly calculated for a given timestamp. The declination of the sun δ can be aproximated by:

$$\delta = -4092797 \cos(\frac{2\pi}{365}(M + 10)). \quad (5)$$

Where M denotes the day of the year. The constant 0.4092797 represents the maximum declination angle of the sun in radians, namely, 23.45 degrees.

Next, for a time τ in hours rooted at UTC+0, longitude of the sun is calculated as:

$$\lambda = \frac{360\tau}{24} \quad (6)$$

Next, observe that for a given measured sun elevation e , and sun SP (δ, λ) , the only points on the globe whos sun elevation is measured as e form a circle around the SP. this circle represents all possible locations in which such an elevation e could be measured.

E. Geolocation via Multi-Image Constraint Solving

[TODO: This is the novel contribution of your pipeline. Explain:

- **For a single image with timestamp t_k and predicted elevation θ_k , equation (4) yields a locus of (ϕ, λ) pairs (a curve on the Earth's surface).**
- **Two or more such curves (from different times) intersect at the true location.**
- **How you solve for the intersection (e.g. least-squares optimisation, grid search, or Levenberg–Marquardt on the residual $\sum_k (g(\phi, \lambda, t_k) - \hat{\theta}_k)^2$).**
- **Any ambiguities (e.g. hemisphere ambiguity) and how you handle them.**

A small diagram showing the intersecting loci on a map would strengthen this sub-section.]

We have seen we can find a circle of possible locations for where a measured elevation can occur. If we then take multiple easurements from multiple times throughout the year in the same area, we can obtain multiple of these sets of possible locations. The intersection of these curves can be taken to find two unique points on the globe where the set of measurements must have come from.

Two unique points is as good as this method can do. These 2 points represent reflections around the equator. Narrowing the potential points further is trivial in many cases. If one of the points is over the ocean, and the image we are trying to geolocate is on dry land, we can discard it, for example.

In many cases we can constrain our potential location set by timezone. If the timestamp of the images includes a timezone such as NZST, only the points of the circles within that timezone need to be considered. Many times this rules out one of the mirrored points as it does not fall within the target timezone.

The geolocation estimate $(\hat{\phi}, \hat{\lambda})$ minimises:

$$(\hat{\phi}, \hat{\lambda}) = \arg \min_{\phi, \lambda} \sum_{k=1}^K (g(\phi, \lambda, t_k) - \hat{\theta}_k)^2. \quad (7)$$

F. Implementation Details

[TODO: List hyperparameters and software stack concisely (a table is clean): backbone, input resolution, batch size, optimiser (Adam, SGD), learning rate schedule, number of epochs, hardware (GPU model), framework (PyTorch / TensorFlow), sun-position library (pysolar, pvlib, or custom). This lets readers reproduce your work.]

TABLE I: Implementation hyperparameters

Parameter	Value
Backbone	EfficientNet (ImageNet-1K pretrained)
Input resolution	224×224
Batch size	16
Optimiser	Adam ($lr = 1e - 4$)
Epochs	30
Hardware	NVIDIA GeForce RTX 4060 Ti

IV. EXPERIMENTS AND RESULTS

A. Evaluation Metrics

[TODO: Define all metrics you report:

- **Elevation MAE / RMSE (degrees): primary CNN accuracy metric.**
- **Geolocation error (km): great-circle distance between predicted and ground-truth coordinates. State whether you report mean, median, or top-X% accuracy.**

]

We evaluate the CNN on a evaluation set of 1,000 images with ground truth labels. The evaluation metric is measuerd as MAE in degrees.

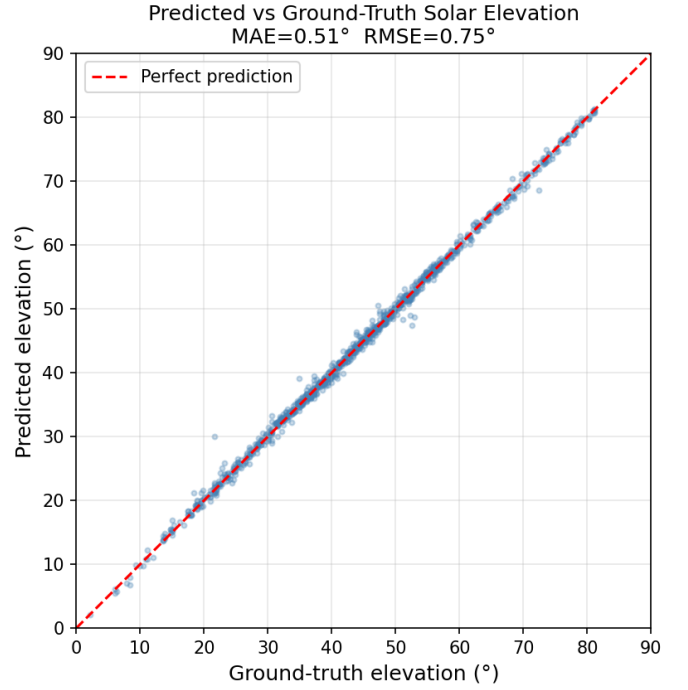


Fig. 2: Ground truth elevation vs. predicted elevation

We also evaluate the geolocation pipeline using the distance between prediction and ground truth in Killometers.

B. CNN Solar Elevation Estimation

[TODO: Report test-set MAE and RMSE. Compare with:

- **A naive baseline (e.g. predict mean training elevation for all images).**
- **Other backbone choices if you ablated them.**

Include a scatter plot of predicted vs. ground-truth elevation angles (Fig. 2) and a residual distribution histogram.]

We evaluated the models predicted elevation with ground truth elevation (Fig. 2) and see a linear result throught evevation. There is minimal bias at high or low angles. This shows that the model has successfully generalized.

We also plot a residual histogram of error (pred - GT) (Fig. 3). There is a minimal positave bias of 0.07 degrees with a very tight spread. We see that 99.8% of images fall within ± 5 degrees, and 98.3% images fall within ± 2 degrees (Tab. II) of the true elevation.

TABLE II: Elevation estimation accuracy (test set)

Metric	Result
MAE	0.511°
RMSE	0.753°
Mean bias	0.069°
Within $\pm 2^\circ$	98.3%
Within $\pm 5^\circ$	99.8%

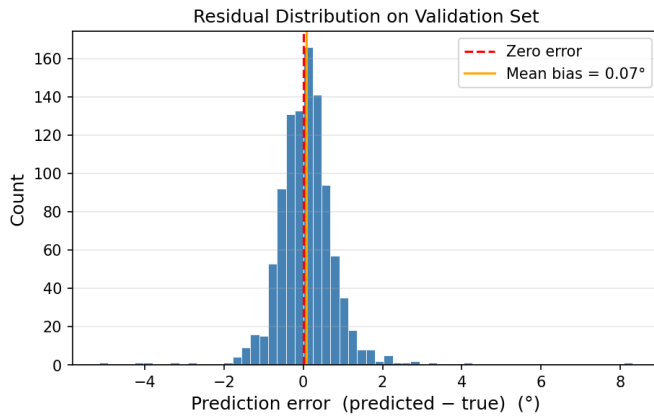


Fig. 3: CNN elevation estimation performance on the test set.

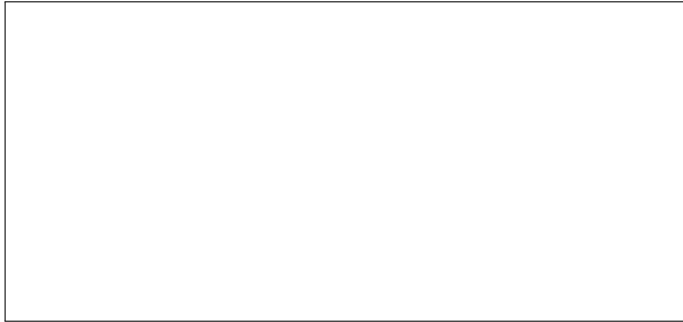


Fig. 4: Example geolocation results. $\star$ = ground truth; $\bullet$ = predicted location. [TODO: Replace placeholder with an actual map figure.]

C. Geolocation Accuracy

[TODO: Report geolocation results. Suggested structure:

- Table or figure showing median / mean error (km) as a function of the number of input images K (e.g. $K = 2, 3, 4, 6$).
- A world map or local map showing predicted vs. true locations for representative test sequences (Fig. 4).
- Discuss upper-bound performance using ground-truth elevations instead of CNN estimates, to separate CNN error from model error.

]

D. Ablation Study

[TODO: Optional but strongly recommended. Test the effect of:

- Number of images K on geolocation error.
- Degree of fine-tuning (head only vs. last block vs. full network).
- Input resolution.
- Time-of-day distribution in training data.

]

E. Qualitative Analysis

[TODO: Show 2–3 interesting examples (success cases and failure cases). For each, display the image, the

predicted elevation, and the resulting location estimate. Discuss what visual cues the model appears to be using (shadow length/direction, sky colour gradient, haze) using a Grad-CAM or similar visualisation if possible.]

V. DISCUSSION

[TODO: Address the following points:

- What worked well? Which conditions (clear sky, strong shadows, golden hour) produce the most accurate elevation estimates?
- What failed and why? Overcast skies, indoor/artificial lighting, extreme latitudes, near-solstice dates (low sun-movement rate), timestamp errors.
- Error budget: How much of the final geolocation error is attributable to CNN elevation error vs. inherent geometric ambiguity?
- Practical considerations: Minimum number of images needed for reliable geolocation; effect of image spacing in time; hemisphere ambiguity.
- Comparison to GPS accuracy: Put your error (km) in context.

]

VI. CONCLUSION

[TODO: Write 2–3 paragraphs:

- 1) Restate the problem and your key contributions (1–2 sentences each, do not copy the abstract verbatim).
- 2) Your best result in one sentence (e.g. “With four co-located images our system achieves a median geolocation error of X km without any landmark database or prior map knowledge.”).
- 3) Future work: fine-tuning on more diverse data, adding moon/star cues for night images, uncertainty estimation, real-time deployment.

ACKNOWLEDGEMENTS

The author thanks Richard Green for project supervision and feedback. [TODO: Add any other acknowledgements (dataset providers, compute resources, etc.)]

REFERENCES

- [1] O. Russakovsky *et al.*, “ImageNet Large Scale Visual Recognition Challenge,” *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.
- [2] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE CVPR*, pp. 770–778, 2016.
- [3] J. Hays and A. A. Efros, “IM2GPS: estimating geographic information from a single image,” in *Proc. IEEE CVPR*, 2008.
- [4] T. Weyand, I. Kostrikov, and J. Philbin, “PlaNet – photo geolocation with convolutional neural networks,” in *Proc. ECCV*, 2016.
- [5] I. Reda and A. Andreas, “Solar position algorithm for solar radiation applications,” *Solar Energy*, vol. 76, no. 5, pp. 577–589, 2004.
- [6] J.-F. Lalonde, A. A. Efros, and S. G. Narasimhan, “Estimating the natural illumination conditions from a single outdoor image,” *Int. J. Comput. Vis.*, vol. 98, no. 2, pp. 123–145, 2012.
- [7] [TODO: Add a relevant image forensics / provenance citation.]